Industrial Internship Report on

"FactoryWatch Lite: Full Stack Machine Monitoring Dashboard"

Prepared by

Md Faiz

CT Group of Institutions, Shahpur

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was FactoryWatch Lite, a full stack machine monitoring dashboard built with Node.js, Express, SQLite and Chart.js. It collects temperature, vibration and power readings from machines, stores them in a database, shows them live on a web dashboard and raises alerts when a reading crosses a safe limit.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface....3
2 Introduction....5
2.1 About UniConverge Technologies Pvt Ltd....5
2.2 About upskill Campus....9
2.3 Objective....11
2.4 Reference....11
2.5 Glossary....11
3 Problem Statement....13
4 Existing and Proposed solution....14
5 Proposed Design/ Model....14
5.1 High Level Diagram (if applicable)....15
5.2 Low Level Diagram (if applicable)....15
5.3 Interfaces (if applicable)....16
6 Performance Test....18
6.1 Test Plan/ Test Cases....18
6.2 Test Procedure....19
6.3 Performance Outcome....19
7 My learnings....20
8 Future work scope....20
9 Appendix: Source Code....21

1 Preface

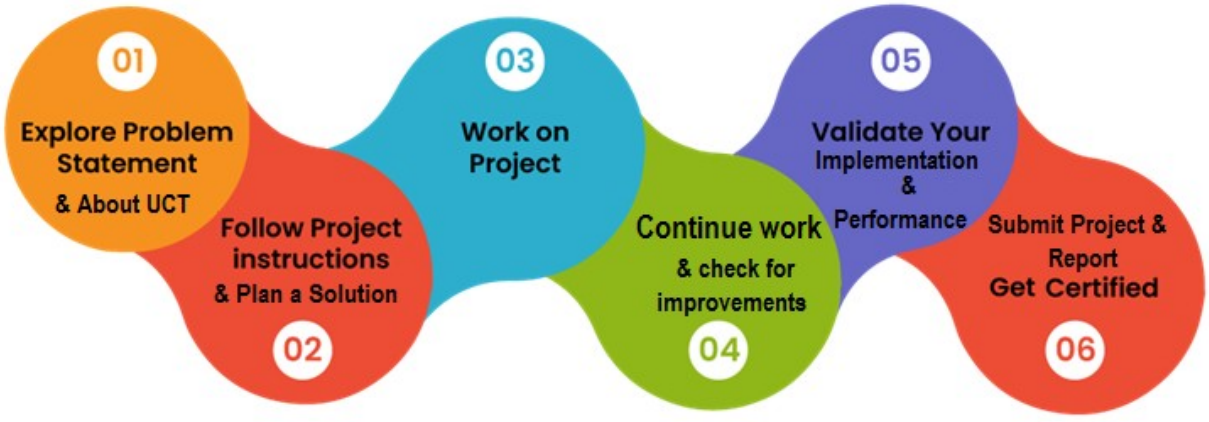
This report summarises the six-week Industrial Internship that I, Md Faiz, a student of CT Group of Institutions, Shahpur, completed with upskill Campus and The IoT Academy in association with UniConverge Technologies Pvt Ltd (UCT). In these six weeks I studied the problem statement, planned a solution, built a working full stack application, tested it and documented the work in this report.

A relevant internship is an important step in career development. Classroom learning gives theory, but an internship shows how the same concepts are used to solve real industrial problems, with deadlines, constraints and testing. It also improves communication, teamwork and confidence, which companies look for during hiring.

My project is FactoryWatch Lite, a full stack machine monitoring dashboard. Many factories check machine health manually and learn about a fault only after a breakdown. FactoryWatch Lite collects temperature, vibration and power readings, stores them in a database, shows them live on a web dashboard and raises an alert when a reading crosses a safe limit.

USC and UCT gave me the opportunity to work on an industry-oriented problem statement, learn from industry experts and use modern technologies such as REST APIs, databases and IoT concepts. The internship became a practical bridge between my college studies and the needs of the industry.

The programme was planned in six weeks: exploring the problem statement and the company, following the project instructions and planning the solution, working on the project, continuing the work with improvements, validating the implementation, and finally submitting the project and report to get certified. The figure below shows this flow.



During this internship I improved my skills in JavaScript, Node.js, Express, SQL and web dashboards. I learned how to design REST APIs, test an application against constraints and write clear technical

documentation. Overall it was a very good experience and it made me more confident about working on real projects.

I sincerely thank the upskill Campus and UniConverge Technologies teams for this opportunity and their guidance, the faculty members and management of CT Group of Institutions, Shahpur for their support, and my family and friends who encouraged me throughout the internship.

To my juniors and peers: start early, choose a problem you are curious about and keep building a little every day. Do not wait for a perfect idea; a working project that you fully understand is worth more than a big idea that is never finished.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



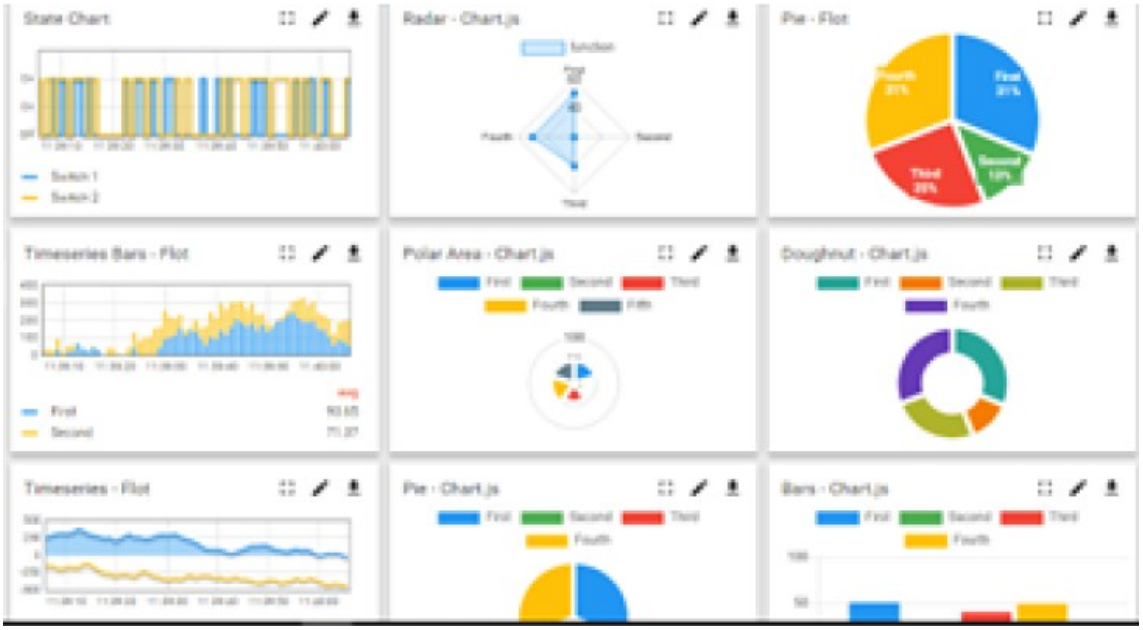
i. UCT IoT Platform (UCT Insight)

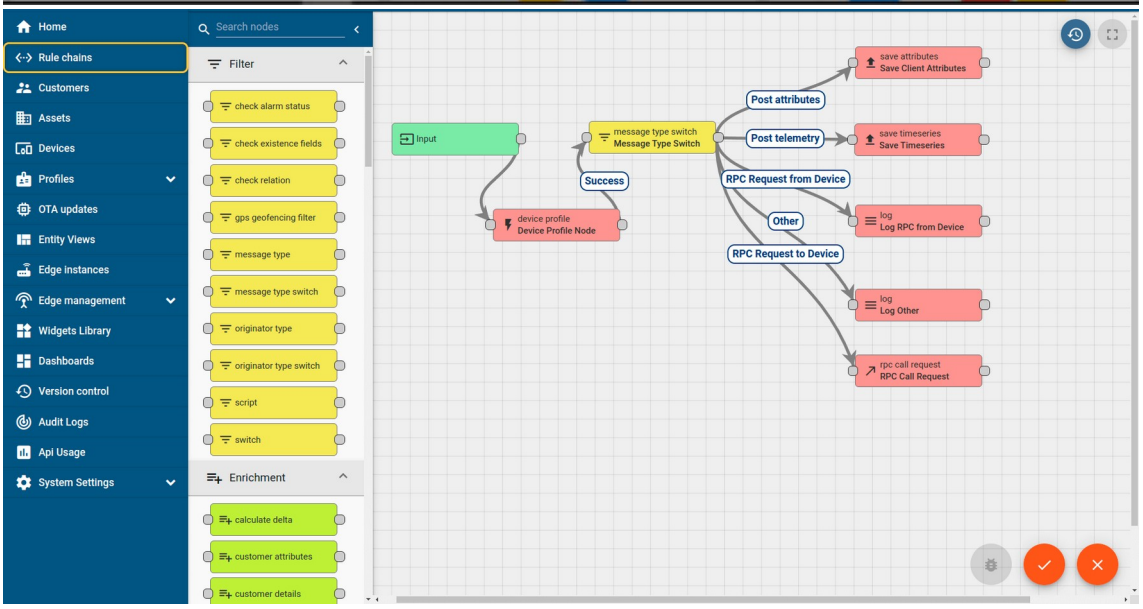
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

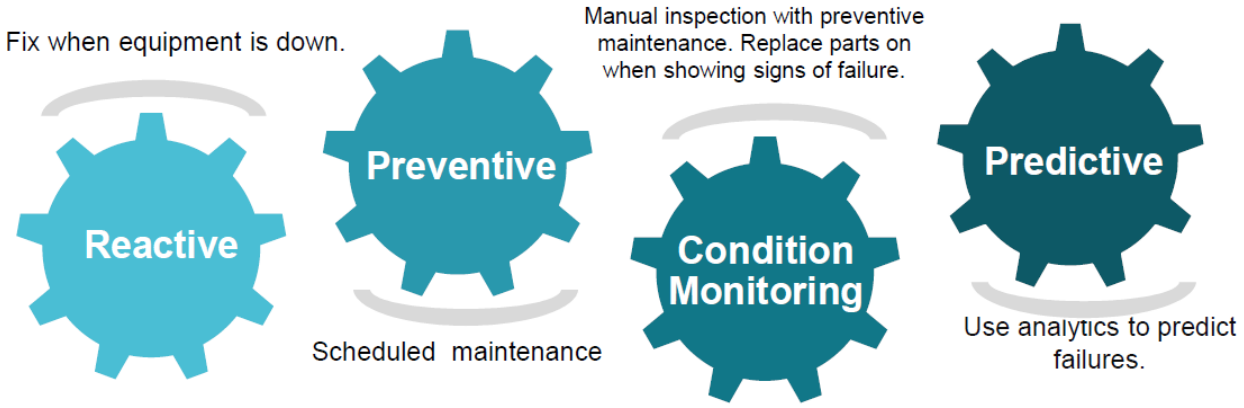


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

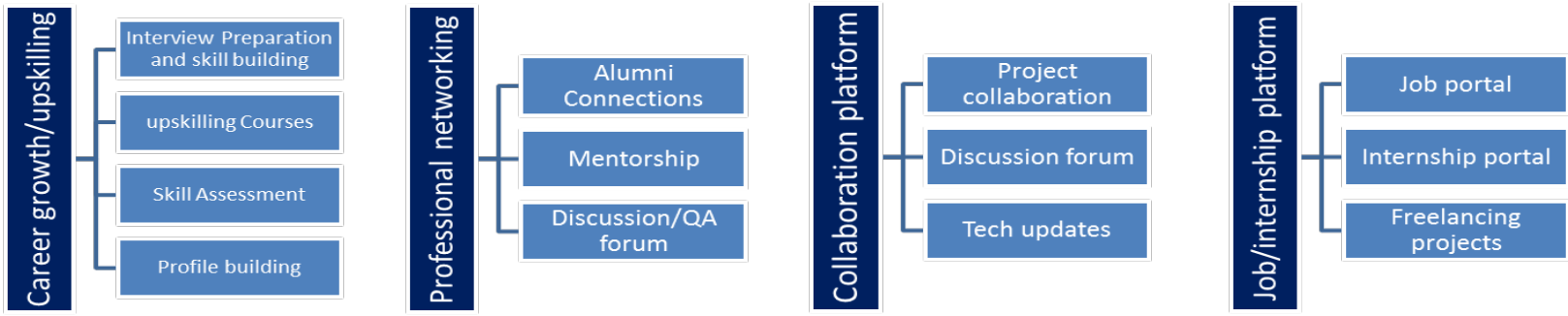
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.


Career growth/upskilling
Interview Preparation and skill building
upskilling Courses
Skill Assessment
Profile building
Professional networking
Alumni Connections
Mentorship
Discussion/QA forum
Collaboration platform
Project collaboration
Discussion forum
Tech updates
Job/internship platform
Job portal
Internship portal
Freelancing projects

2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] Express.js documentation, https://expressjs.com/

[2] SQLite documentation, https://www.sqlite.org/docs.html

[3] Chart.js documentation, https://www.chartjs.org/docs/

[4] Node.js documentation, https://nodejs.org/en/docs

[5] better-sqlite3 library, https://github.com/WiseLibs/better-sqlite3

2.6 Glossary

Terms	Acronym
IoT	Internet of Things
API	Application Programming Interface
REST	Representational State Transfer

OEE	Overall Equipment Effectiveness
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol

3 Problem Statement

In the assigned problem statement, I had to build a complete industrial application within six weeks.

Small and medium factories usually monitor machine health through manual rounds and paper logs. Abnormal temperature, vibration or power use is noticed late, which leads to unplanned downtime, repair cost and production loss. Commercial SCADA and IIoT platforms solve this but are often too costly and complex for such units. The problem is to design a low-cost, easy-to-deploy web application that collects machine sensor data, stores it, shows it in real time and automatically alerts the operator when a parameter crosses its safe limit.

Objectives: (1) accept machine readings over HTTP, (2) store the history in a database, (3) display live values and trends on a dashboard, and (4) generate alerts with a rule engine.

4 Existing and Proposed solution

Existing solutions and limitations: Existing solutions are manual logbooks, spreadsheets and commercial SCADA/IIoT platforms. Manual methods are slow and error-prone, and faults are found late. Commercial platforms are powerful but expensive and need a long setup, which small units cannot afford.

Proposed solution: FactoryWatch Lite is a lightweight full stack web application (Node.js, Express, SQLite, Chart.js). Machines or IoT devices send readings to a REST API, the server stores them and checks them against limits, and a browser dashboard shows live values, a trend graph and an alert list.

Value addition: Low cost, setup in two commands (npm install, npm start), live monitoring with 2 second refresh, automatic alerts, and an open REST API so real IoT devices can be connected later. The same design can be extended with login, MQTT and predictive maintenance.

Aspect	Manual logs / spreadsheets	Commercial SCADA / IIoT	FactoryWatch Lite
Cost	Low	High	Low (open-source tools)
Real-time view	No	Yes	Yes (2 second refresh)
Automatic alerts	No	Yes	Yes (rule engine)
Setup effort	Daily manual work	Complex	Simple (two commands)
Extensibility	Poor	Depends on vendor	Open REST API

4.1 Code submission (Github link)

https://github.com/mdfaiz/factorywatch-lite

4.2 Report submission (Github link)

https://github.com/mdfaiz/factorywatch-lite/blob/main/Internship_Report_Md_Faiz.docx

5 Proposed Design/ Model

FactoryWatch Lite follows a three-tier design. The data tier is a SQLite database. The application tier is a Node.js/Express server that exposes REST APIs, runs the rule engine and contains a sensor simulator. The presentation tier is a browser dashboard that polls the API every two seconds. The flow is: a reading arrives from a device or the simulator, it is saved in the database, the rule engine checks the limits, an

alert is stored if a limit is crossed, and the dashboard fetches and shows the latest values, graph and alerts.

5.1 High Level Diagram (if applicable)

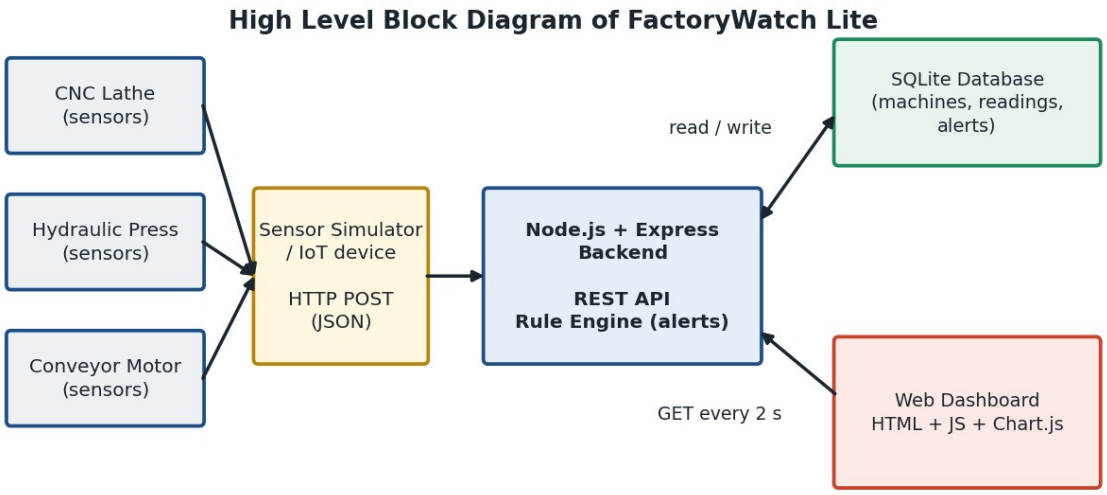


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

The database has three tables. machines stores the machine name and location. readings stores temperature, vibration, power and a timestamp for each machine. alerts stores the alert message and time. The REST API reads and writes these tables, and the rule engine inserts a row in alerts whenever a limit is crossed.

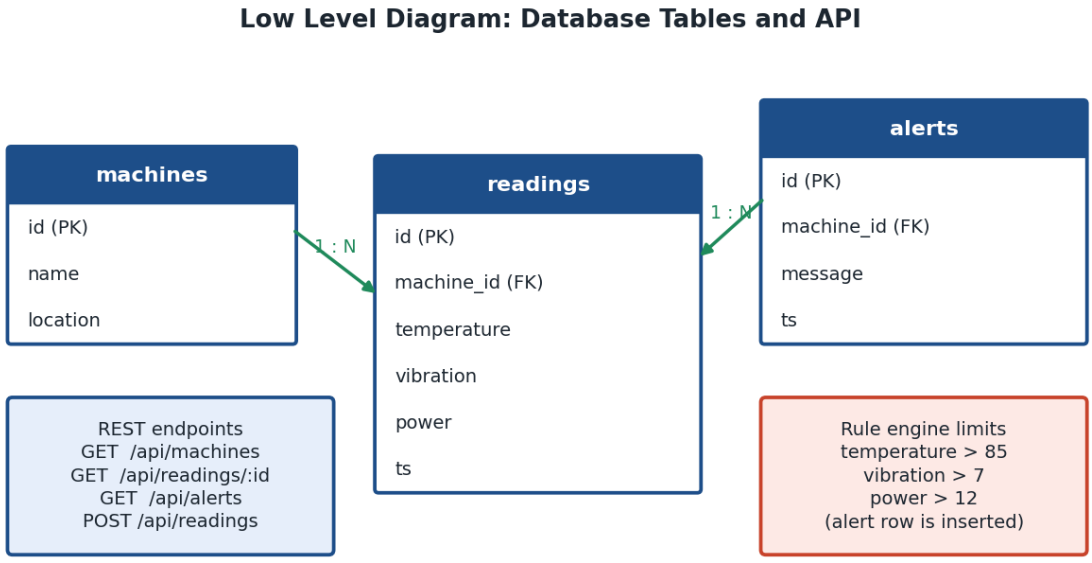


Figure 2: LOW LEVEL DIAGRAM (DATABASE TABLES AND API)

5.3 Interfaces (if applicable)

All interfaces use HTTP with JSON. Devices send data with POST /api/readings and the dashboard reads data with GET requests. The flowchart below shows what happens to every reading. A sample request body is:

```
{"machine_id": 1, "temperature": 92, "vibration": 3.2, "power": 8.5}
```

Method	Endpoint	Purpose
GET	/api/machines	Latest reading of every machine
GET	/api/readings/:id	Last 30 readings of one machine
GET	/api/alerts	15 most recent alerts
POST	/api/readings	Add a reading (400 if the data is invalid)

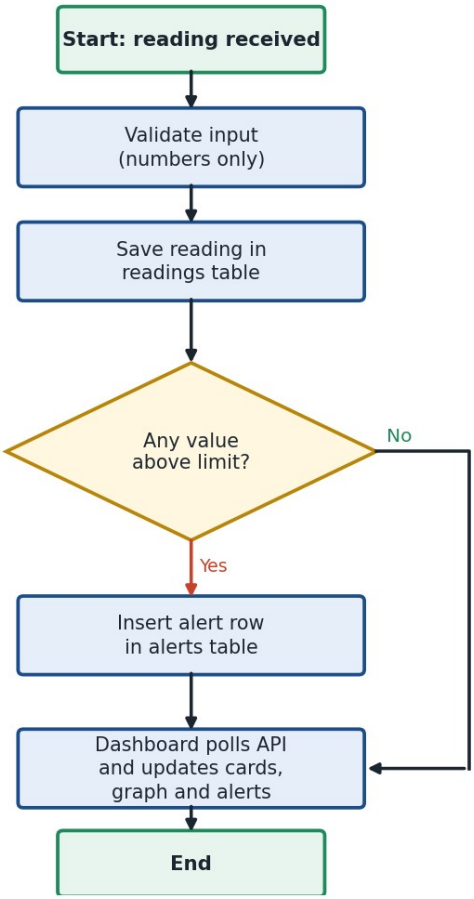


Figure 3: FLOWCHART OF READING PROCESSING AND ALERTING

6 Performance Test

Performance testing shows whether FactoryWatch Lite can work reliably in a real factory environment and not only as an academic demo.

Constraints identified: (1) response time of the dashboard, (2) growth of the database as readings keep arriving, (3) correctness of alerts, (4) memory and CPU use on a low-cost computer, (5) robustness against invalid input, and (6) security.

How the constraints were handled: the API returns only the latest 30 readings per machine, so every response stays small; queries use LIMIT; the rule engine uses simple threshold comparisons; invalid POST data is rejected with HTTP 400; and the application uses only two dependencies (Express and better-sqlite3).

Test results: the test cases and expected outcomes for these constraints are given in sections 6.1 and 6.3.

Note: For this software project the main constraints were speed (response time), memory use and alert accuracy.

Not tested: Some constraints could not be fully tested in six weeks: a very large number of machines, continuous multi-day operation and security attacks. With many machines the database will grow fast and queries will slow down, so an index on (machine_id, id), archiving of old data and a move to MySQL or PostgreSQL are recommended. For real deployment, login or an API key and HTTPS should be added.

6.1 Test Plan/ Test Cases

ID	Test case	Input / action	Expected result
TC1	Server start	npm start	Server runs on port 3000, tables are created and 3 machines are added
TC2	Live data	Open the dashboard	3 machine cards show values that refresh every 2 seconds
TC3	Normal reading	POST temp 70, vibration 3, power 8	HTTP 201 and no alert is created
TC4	High temperature	POST temp 92	HTTP 201, alert 'temperature high' is created, card turns red
TC5	High vibration / power	POST vibration 8.5, power 14	An alert is created for each crossed limit
TC6	Invalid input	POST with text value or missing field	HTTP 400 with an error message, server keeps running
TC7	Machine selection	Click a machine card	Graph shows the last 30 readings of that machine
TC8	Data persistence	Restart the server	Old readings and alerts are still available (factory.db)

6.2 Test Procedure

1. Install Node.js, open the project folder and run: npm install

2. Start the server with: npm start, then open http://localhost:3000 in a browser.

3. Watch the dashboard for a minute to check live refresh and the simulator alerts (TC1, TC2, TC7).

4. Send test readings from another terminal with curl (TC3 to TC6):

```
curl -X POST http://localhost:3000/api/readings -H "Content-Type: application/json" -d
'{"machine_id":1,"temperature":92,"vibration":3,"power":8}'
```

5. Stop and restart the server and confirm that old data is still shown (TC8).

6.3 Performance Outcome

The table lists the design targets and the expected result for each constraint. Exact timings depend on the hardware used for testing.

Constraint	How it is handled	Expected outcome
Response time	Small JSON responses, LIMIT 30 on queries	API responds within milliseconds on a normal PC (target below 200 ms)
Alert accuracy	Simple threshold rules (85, 7, 12)	Every reading above a limit creates an alert; no alert below the limits
Memory / CPU	Light stack, only two dependencies	Low usage on a normal PC; suitable for a low-cost computer
Robustness	Input validation on POST	Invalid data gives HTTP 400 and the server keeps running
Persistence	SQLite database file	Data survives a server restart

7 My learnings

I learned how a complete full stack application is built: designing database tables, writing REST APIs in Node.js and Express, creating a live dashboard with JavaScript and Chart.js, and testing the system against real-world constraints. I also learned how to break a big problem into weekly tasks, debug errors and write technical documentation. These skills will help my career growth in software development, IoT and data-driven industrial applications.

8 Future work scope

Because of the six-week time limit, the following ideas could not be implemented and can be taken up in future:

1. Login and role-based access (operator, manager) with HTTPS.

2. MQTT support so that real IoT devices can publish data directly.

3. Email or SMS notification when an alert occurs.

4. OEE calculation and downtime reports.

5. Predictive maintenance with machine learning to estimate remaining useful life of machines.

6. Moving to MySQL or PostgreSQL with indexes and old-data archiving for large factories.

7. A mobile-friendly app or push notifications.

9 Appendix: Source Code

Complete source code of the FactoryWatch Lite project is included below so that the whole project is available in this single file.

9.1 package.json

```
{
  "name": "factorywatch-lite",
  "version": "1.0.0",
  "description": "Full stack machine monitoring dashboard (Node.js + Express + SQLite + Chart.js)",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "better-sqlite3": "^11.3.0",
    "express": "^4.19.2"
  }
}
```

9.2 server.js (Backend)

```
const express = require("express");
const Database = require("better-sqlite3");
const path = require("path");

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

// ---------- Database ----------
const db = new Database("factory.db");
db.exec(`
CREATE TABLE IF NOT EXISTS machines (id INTEGER PRIMARY KEY, name TEXT, location TEXT);
CREATE TABLE IF NOT EXISTS readings (
  id INTEGER PRIMARY KEY AUTOINCREMENT, machine_id INTEGER,
  temperature REAL, vibration REAL, power REAL, ts DATETIME DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS alerts (
  id INTEGER PRIMARY KEY AUTOINCREMENT, machine_id INTEGER,
  message TEXT, ts DATETIME DEFAULT CURRENT_TIMESTAMP);
`);
if (db.prepare("SELECT COUNT(*) c FROM machines").get().c === 0) {
  const ins = db.prepare("INSERT INTO machines (name, location) VALUES (?, ?)");
  ["CNC Lathe", "Hydraulic Press", "Conveyor Motor"].forEach((n, i) => ins.run(n, "Line " + (i + 1)));
}

// ---------- Rule engine ----------
const LIMITS = { temperature: 85, vibration: 7, power: 12 };
function saveReading({ machine_id, temperature, vibration, power }) {
  db.prepare("INSERT INTO readings (machine_id, temperature, vibration, power) VALUES (?,?,?,?)")
    .run(machine_id, temperature, vibration, power);
  const data = { temperature, vibration, power };
  for (const k in LIMITS) {
    if (data[k] > LIMITS[k]) {
      db.prepare("INSERT INTO alerts (machine_id, message) VALUES (?,?)")
        .run(machine_id, `${k} high: ${data[k].toFixed(1)} (limit ${LIMITS[k]})`);
    }
  }
}

// ---------- REST API ----------
app.get("/api/machines", (req, res) => {
  const rows = db.prepare(`
    SELECT m.*, r.temperature, r.vibration, r.power, r.ts FROM machines m
    LEFT JOIN readings r ON r.id = (SELECT MAX(id) FROM readings WHERE machine_id = m.id)`).all();
  res.json(rows);
});
```

```
app.get("/api/readings/:id", (req, res) => {
  const rows = db.prepare("SELECT * FROM readings WHERE machine_id=? ORDER BY id DESC LIMIT 30")
    .all(req.params.id);
  res.json(rows.reverse());
});
app.get("/api/alerts", (req, res) => {
  res.json(db.prepare(`SELECT a.*, m.name FROM alerts a JOIN machines m ON m.id=a.machine_id
    ORDER BY a.id DESC LIMIT 15`).all());
});
// Real IoT devices can POST here (HTTP protocol)
app.post("/api/readings", (req, res) => {
  const { machine_id, temperature, vibration, power } = req.body;
  if ([machine_id, temperature, vibration, power].some(v => typeof v !== "number"))
    return res.status(400).json({ error: "machine_id, temperature, vibration, power must be numbers" });
  saveReading(req.body);
  res.status(201).json({ ok: true });
});

// ---------- Sensor simulator (replace with real devices) ----------
setInterval(() => {
  db.prepare("SELECT id FROM machines").all().forEach(({ id }) => {
    const spike = Math.random() < 0.08;
    saveReading({
      machine_id: id,
      temperature: 60 + Math.random() * 20 + (spike ? 15 : 0),
      vibration: 2 + Math.random() * 4 + (spike ? 3 : 0),
      power: 6 + Math.random() * 5 + (spike ? 3 : 0),
    });
  });
}, 2000);

app.listen(3000, () => console.log("FactoryWatch Lite running at http://localhost:3000"));
```

9.3 public/index.html (Frontend)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>FactoryWatch Lite</title>
<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.1/chart.umd.min.js"></script>
<style>
  :root { --bg:#eef1f4; --card:#fff; --ink:#1c2630; --mute:#6b7885; --ok:#1f8a5b; --bad:#c8432b; --
line:#d5dbe1; --accent:#1d4e89; }
  * { box-sizing: border-box; }
  body { margin:0; font-family: "Segoe UI", system-ui, sans-serif; background:var(--bg); color:var(--
ink); }
  header { padding:16px 20px; background:var(--ink); color:#fff; }
  header h1 { margin:0; font-size:1.15rem; font-weight:600; }
  header p { margin:2px 0 0; font-size:.85rem; color:#aab6c2; }
  main { max-width:1000px; margin:0 auto; padding:16px; display:grid; gap:16px; }
  .machines { display:grid; grid-template-columns:repeat(auto-fit,minmax(240px,1fr)); gap:12px; }
  .m { background:var(--card); border:1px solid var(--line); border-left:6px solid var(--ok); border-
radius:6px; padding:12px; cursor:pointer; }
  .m.hot { border-left-color:var(--bad); }
  .m.sel { outline:2px solid var(--accent); }
  .m h3 { margin:0; font-size:1rem; } .m small { color:var(--mute); }
  .vals { display:flex; gap:14px; margin-top:8px; font-size:.9rem; }
  .vals b { display:block; font-size:1.2rem; }
  .panel { background:var(--card); border:1px solid var(--line); border-radius:6px; padding:14px; }
  .panel h2 { margin:0 0 10px; font-size:1rem; }
  ul { list-style:none; margin:0; padding:0; } li { padding:7px 0; border-bottom:1px solid var(--line);
font-size:.9rem; }
  li:last-child { border:none; } li span { color:var(--mute); font-size:.8rem; }
  button:focus-visible, .m:focus-visible { outline:3px solid var(--accent); }
</style>
</head>
<body>
<header><h1>FactoryWatch Lite</h1><p>Live machine health, updates every 2 seconds</p></header>
```

```
<main>
  <div class="machines" id="machines"></div>
  <div class="panel"><h2 id="chartTitle">Select a machine</h2><canvas id="chart"
height="110"></canvas></div>
  <div class="panel"><h2>Recent alerts</h2><ul id="alerts"><li>No alerts yet.</li></ul></div>
</main>
<script>
let selected = 1, chart;
const LIM = { temperature: 85, vibration: 7, power: 12 };

async function api(url) { const r = await fetch(url); return r.json(); }

async function refresh() {
  const machines = await api("/api/machines");
  document.getElementById("machines").innerHTML = machines.map(m => {
    const hot = m.temperature > LIM.temperature || m.vibration > LIM.vibration || m.power > LIM.power;
    const f = v => (v == null ? "-" : v.toFixed(1));
    return `<div class="m ${hot ? "hot" : ""} ${m.id === selected ? "sel" : ""}" tabindex="0" data-id="$
{m.id}">
      <h3>${m.name}</h3><small>${m.location} · ${hot ? "Needs attention" : "Normal"}</small>
      <div class="vals"><div><b>${f(m.temperature)}°C</b>Temp</div><div><b>${f(m.vibration)}</
b>Vibration</div><div><b>${f(m.power)} kW</b>Power</div></div></div>`;
  }).join("");
  document.querySelectorAll(".m").forEach(el => el.onclick = () => { selected = +el.dataset.id;
refresh(); });

  const name = (machines.find(m => m.id === selected) || {}).name || "";
  document.getElementById("chartTitle").textContent = name + ": last 30 readings";
  const rows = await api("/api/readings/" + selected);
  const labels = rows.map(r => r.ts.slice(11, 19));
  const ds = (label, key, color) => ({ label, data: rows.map(r => r[key]), borderColor: color,
tension: .3, pointRadius: 0 });
  const data = { labels, datasets: [ds("Temperature (°C)", "temperature", "#c8432b"), ds("Vibration",
"vibration", "#1d4e89"), ds("Power (kW)", "power", "#1f8a5b")] };
  if (!chart) chart = new Chart(document.getElementById("chart"), { type: "line", data, options:
{ animation: false } });
  else { chart.data = data; chart.update(); }

  const alerts = await api("/api/alerts");
  if (alerts.length) document.getElementById("alerts").innerHTML =
    alerts.map(a => `<li><b>${a.name}</b>: ${a.message}<br><span>${a.ts}</span></li>`).join("");
}
refresh(); setInterval(refresh, 2000);
</script>
</body>
</html>
```